

Repaso I1

Clase repaso I

IIC 2133 coordinado

Prof. Yadran Eterovic, Mario
Droguett, Eduardo Bustos

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

Interrogación 1

Objetivos a evaluar en la I1

- ☐ Comprender y comparar algoritmos de ordenación clásicos
- ☐ Modificar algoritmos conocidos para resolver problemas
- ☐ Diseñar algoritmos usando técnicas e ideas estudiadas
- ☐ Demostrar correctitud y complejidad de algoritmos
- ☐ Comprender el funcionamiento de EDDs de árboles
- ☐ Comparar estructuras basadas en árboles

Varios objetivos pueden incluirse en cada pregunta

Interrogación 1

Formato de la prueba

- 2 horas de tiempo
- Pool de 4 preguntas para elegir 3
- Cada pregunta incluye un título que describe sus temas
- ¡**SOLO** se entregan 3 preguntas respondidas! físicamente y en Canvas

Nota de la I1: promedio de las 3 preguntas entregadas

Interrogación 1

Material adicional (Torpedo)

- Pueden usar un apunte durante la prueba
- Debe estar escrito a mano
- Una hoja oficio (por ambos lados)
- Sugerencia: incluyan los pseudocódigos vistos

No se aceptarán diapositivas impresas

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

SelectionSort

Lema: *seleccionar de forma ordenada*

Funcionamiento a alto nivel

1. Seleccionar menor elemento
2. Ubicarlo como último elemento en zona *ordenada*
3. Repetir hasta seleccionar **todos** los elementos

Desempeño en casos

- No distingue secuencias por su orden a priori
- Mismo número de comparaciones en todas las secuencias

Complejidad en la secuencia $A[0 \dots n - 1]$

- Versión original: tiempo $\mathcal{O}(n^2)$ y memoria $\mathcal{O}(n)$
- Versión *in place*: tiempo $\mathcal{O}(n^2)$ y memoria $\mathcal{O}(1)$

SelectionSort

input : Secuencia $A[0 \dots n-1]$, largo $n \geq 2$

output: $\emptyset$

SelectionSort (A, n):

```
1  for  $i = 0 \dots n-2$  :  
2       $min \leftarrow i$   
3      for  $j = i+1 \dots n-1$  :  
4          if  $A[j] < A[min]$  :  
5               $min \leftarrow j$   
6       $A[i] \rightleftharpoons A[min]$ 
```

Versión *in place* en tiempo $\mathcal{O}(n^2)$

InsertionSort

Lema: *insertar de forma ordenada*

Funcionamiento a alto nivel

1. Seleccionar el primer elemento no revisado
2. Moverlo hasta su posición correcta
3. Repetir hasta ubicar **todos** los elementos

Desempeño en casos

- Como revisa si el elemento está bien insertado...
- ... *se da cuenta* cuando un elemento está ordenado

Complejidad en la secuencia $A[0 \dots n - 1]$

- Mejor caso: secuencia ordenada
 - Versión *in place*: tiempo $\mathcal{O}(n)$ y memoria $\mathcal{O}(1)$
- Todos los demás casos:
 - Versión *in place*: tiempo $\mathcal{O}(n^2)$ y memoria $\mathcal{O}(1)$

InsertionSort

input : Secuencia $A[0 \dots n - 1]$, largo $n \geq 2$

output: $\emptyset$

InsertionSort (A, n):

```
1  for  $i = 1 \dots n - 1$  :  
2       $j = i$   
3      while  $(j > 0) \wedge (A[j] < A[j - 1])$  :  
4          Intercambiar  $A[j]$  con  $A[j - 1]$   
5           $j = j - 1$ 
```

Versión *in place*: mejor caso $\mathcal{O}(n)$, e.o.c. $\mathcal{O}(n^2)$

MergeSort

Lema: *mezclar mitades ordenadas*

Funcionamiento a alto nivel

1. Recursivamente dividir secuencia en mitades
2. Caso base: secuencias de largo 1
3. Mezclar ordenadamente subsecuencias ordenadas (Merge)

Desempeño

- Profundidad de llamados recursivos **siempre** logarítmica
- Mezcla lineal en todos los casos

Complejidad en la secuencia $A[0 \dots n-1]$

- tiempo $\mathcal{O}(n \log(n))$ y memoria $\mathcal{O}(n)$

MergeSort

input : Secuencia A

output: Secuencia ordenada B

MergeSort (A):

```
1  if  $|A| = 1$  : return  $A$ 
2  Dividir  $A$  en  $A_1$  y  $A_2$ 
3   $B_1 \leftarrow \text{MergeSort}(A_1)$ 
4   $B_2 \leftarrow \text{MergeSort}(A_2)$ 
5   $B \leftarrow \text{Merge}(B_1, B_2)$ 
6  return  $B$ 
```

Merge(A, B):

```
1  Iniciar  $C$  vacía
2  while  $|A| > 0 \wedge |B| > 0$  :
3      if  $A[1] \leq B[1]$  :
4           $e \leftarrow \text{Extraer}(A[1])$ 
5      else:
6           $e \leftarrow \text{Extraer}(B[1])$ 
7      Insertar  $e$  al final de  $C$ 
8  Concatenar  $C$  con la
   secuencia restante
9  return  $C$ 
```

Tiempo $\mathcal{O}(n \log(n))$ y memoria $\mathcal{O}(n)$

QuickSort

Lema: *ordenar pivotes de forma recursiva*

Funcionamiento a alto nivel

1. Recursivamente ordenar un elemento arbitrario (pivote)
2. Caso base: secuencias de largo 0
3. No requiere mezclar: Partition ordena elementos

Desempeño

- Depende de la elección del pivote
- De antemano no podemos anticipar el desempeño: probabilístico

Complejidad en la secuencia $A[0 \dots n - 1]$

- Mejor caso y promedio
 - Versión *in place*: tiempo $\mathcal{O}(n \log(n))$ y memoria $\mathcal{O}(1)$
- Peor caso: pivote *mágicamente malo* siempre
 - Versión *in place*: tiempo $\mathcal{O}(n^2)$ y memoria $\mathcal{O}(1)$

QuickSort

input : Secuencia

$A[0, \dots, n-1]$,

índices i, f

output: $\emptyset$

QuickSort (A, i, f):

```
1  if  $i \leq f$  :  
2       $p \leftarrow \text{Partition}(A, i, f)$   
3      Quicksort( $A, i, p-1$ )  
4      Quicksort( $A, p+1, f$ )
```

Partition (A, i, f):

```
1   $x \leftarrow$  índice aleatorio en  
     $\{i, \dots, f\}$ ;  $p \leftarrow A[x]$   
2   $A[x] \rightleftharpoons A[f]$   
3   $j \leftarrow i$   
4  for  $k = i \dots f-1$  :  
5      if  $A[k] < p$  :  
6           $A[j] \rightleftharpoons A[k]$   
7           $j \leftarrow j+1$   
8   $A[j] \rightleftharpoons A[f]$   
9  return  $j$ 
```

Caso promedio y mejor caso: tiempo $\mathcal{O}(n \log(n))$ y memoria $\mathcal{O}(1)$

Mejoras en Quicksort

- Para sub-secuencias pequeñas (e.g. $n \leq 20$) podemos usar InsertionSort
 - A pesar de no tener una complejidad mejor
 - Eso es solo cuando hablamos asintóticamente
 - En la práctica, en instancias pequeñas tiene mejor desempeño
 - No olvidar que Quicksort es **recursivo**... eso cuesta recursos
- Usar la mediana de tres elementos como pivote
 - *Informar* la elección de pivote
 - Dado A , escogemos 3 elementos $A[k_1], A[k_2], A[k_3]$
 - En $\mathcal{O}(1)$ encontramos la mediana entre ellos
- Particionar la secuencia en 3 sub-secuencias: menores, iguales y mayores
 - Mejora para caso con datos repetidos
 - Evita que Partition particione innecesariamente sub-secuencias en que todos los valores son iguales

Heaps binarios

Definición

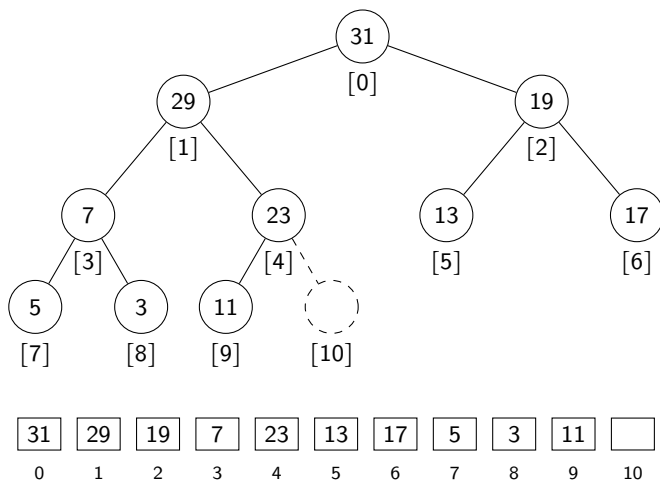
Un **heap binario** H es un árbol binario tal que

- $H.left$ y $H.right$ son heaps binarios
- $H.value > H.left.value$
- $H.value > H.right.value$

A estas condiciones les llamamos **propiedad de heap**

Esta es la definición de un **MAX heap**

Heaps binarios: representación compacta



Heaps binarios

Aspectos esenciales

- Propiedad de heap

descendientes $<$ nodo $<$ ancestros

- Se asegura **balance** mediante sus operaciones

- Inserción al final del arreglo, reubicando (SiftUp)
- Extracción de la raíz, reemplazando con el último y reubicando (SiftDown)
- Actualización de prioridad (SiftUp)

Operaciones logarítmicas en la cantidad de nodos

Heaps binarios

Operaciones

- Inserción: agregar un nuevo elemento
 - Se inserta al final del arreglo
 - Se reubica con `SiftUp`
- Extracción: sacar y retornar el más prioritario
 - Se saca el primer elemento y se reemplaza con el último
 - Se reubica la nueva raíz con `SiftDown`
- Actualización: aumentarle la prioridad a un nodo
 - Se reubica el nodo con `SiftUp`

Todas estas son operaciones $\mathcal{O}(\log(n))$

Los heaps no se usan para buscar,
sino para informar el elemento más prioritario

Heaps binarios

Orientaciones para el estudio

- ☐ Comprender la representación compacta de heaps
- ☐ Comprender operaciones y su uso para mantener una cola de prioridad
- ☐ Considerar casos borde: ¿qué pasa si la prioridad es el orden de llegada? ¿Qué se puede hacer más rápido y cómo?

Incluyan en el formulario el pseudocódigo de los métodos de heaps

Orden lineal

Aspectos esenciales de los algoritmos de orden lineal

- Operan sobre naturales (índices para un arreglo)
- Counting Sort permite ordenar naturales de un conjunto acotado
- La idea se generaliza a Radix para ordenar palabras cualesquiera
 - Cada símbolo se asocia con un natural
 - Requiere un algoritmo estable como Counting Sort

Estos algoritmos ordenan n datos en tiempo $\mathcal{O}(n)$
si la cantidad de símbolos diferentes es $\mathcal{O}(n)$

Orden lineal

Orientaciones para el estudio

- ☐ Comprender cuándo se pueden ocupar los algoritmos lineales
- ☐ Comprender pseudocódigo de Counting Sort y Radix para potenciales modificaciones

Complejidad de algoritmos de ordenación

Resumimos los resultados de complejidad por caso hasta el momento

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria adicional
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Merge Sort	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n)$
Quick Sort	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Heap Sort	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n \log(n))$	$\mathcal{O}(n \log(n))$	$\mathcal{O}(1)$

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

Estrategias algorítmicas

Además de estudiar algoritmos iterativos, vimos dos ejemplos recursivos de la estrategia **dividir para conquistar**

A saber,

- MergeSort
- QuickSort

Dividir para conquistar

La estrategia sigue los siguientes pasos

1. Dividir el problema original en dos (o más) **sub-problemas** del mismo tipo
2. Resolver **recursivamente** cada sub-problema
3. Encontrar solución al problema original **combinando** las soluciones a los sub-problemas

Los sub-problemas son instancias más pequeñas del problema a resolver

Un ejemplo adicional: Búsqueda binaria

El algoritmo de **búsqueda binaria** está basado en la estrategia dividir para conquistar

BSearch (A, x, i, f):

```
1  if  $f < i$  : return -1
2   $m \leftarrow \left\lfloor \frac{i+f}{2} \right\rfloor$ 
3  if  $A[m] = x$  : return  $m$ 
4  if  $A[m] > x$  :
5      return BSearch ( $A, x, i, m-1$ )
6  return BSearch ( $A, x, m+1, f$ )
```

Recordar: ciertos algoritmos D.P.C. no resuelven todos los subproblemas

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

Diccionarios

Definición

Un **diccionario** es una estructura de datos con las siguientes operaciones

- **Asociar** un valor a una llave
- **Actualizar** el valor asociado a una llave
- **Obtener** el valor asociado a una llave
- En ciertos casos, **eliminar** de la estructura una asociación llave-valor

Propiedad de búsqueda: valores ordenados en

hijo izquierdo < padre < hijo derecho

Objetivo central: búsqueda eficiente

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

Ejemplo 1: Aplicación de algoritmos

Ejercicio (I1 P2 - 2022-2)

Un archivo contiene datos de todos los estudiantes que han rendido cursos del DCC desde 1980 hasta la fecha. El formato cada registro en el archivo es

RUT	primer_apellido	segundo_apellido	nombre
-----	-----------------	------------------	--------

y los registros se encuentran ordenados por RUT.

Proponga el pseudocódigo de un algoritmo para ordenar los registros alfabéticamente, i.e. según (primer_apellido, segundo_apellido, nombre). Especifique qué estructura básica usará (listas o arreglos). Si p es un registro, puede acceder a sus atributos con $p.primer_apellido$, $p.nombre$, etc. Además, puede asumir que todo algoritmo de ordenación visto en clase puede ordenar respecto a un atributo específico.

Ejemplo 1: Aplicación de algoritmos

input : Arreglo $A[0, \dots, n-1]$ e índices i, f

output: Lista de pares de índices L

```
1 FirstLastNameReps ( $A, i, f$ ):  
2 begin  
3    $L \leftarrow$  lista vacía  
4    $k \leftarrow i$   
5    $j \leftarrow i$   
6   for  $m = 1 \dots f$  :  
7     begin  
8       if  $A[m].primer\_apellido = A[k].primer\_apellido$  :  
9         begin  
10           $j \leftarrow m$   
11        end  
12      else:  
13        begin  
14          if  $k < j$  :  
15            begin  
16              añadir a  $L$  el par  $(k, j)$   
17            end  
18           $k \leftarrow m$   
19           $j \leftarrow m$   
20        end  
21    end
```


Ejemplo 1: Aplicación de algoritmos

`FirstLastNameReps` (A, i, f) entrega una lista con los rangos entre los cuales hay repeticiones de primer apellido entre los índices i y f . De forma similar se define la rutina que entrega rangos de repetidos de segundo apellido.

Ejemplo 1: Aplicación de algoritmos

input : Arreglo $A[0, \dots, n-1]$

output: Arreglo ordenado alfabéticamente

```
1 AlphaSort (A):
2 begin
4   MergeSort(A, 0, n-1, primer_apellido)    ▷ A según 1º apellido
6    $F \leftarrow \text{FirstLastNameReps}(A, 0, n-1)$ 
8   for  $(k, j) \in F$  :
9     begin
11      MergeSort(A, k, j, segundo_apellido)
13       $S \leftarrow (A, k, j)$ 
15      for  $(s, t) \in S$  :
16        begin
18          MergeSort(A, s, t, nombre)
19        end
20      end
21 end
```

Ejemplo 1: Aplicación de algoritmos

Ejercicio (I1 P2 - 2022-2)

Determine la complejidad de peor caso de su algoritmo en función del número de estudiantes en el archivo.

Ejemplo 1: Aplicación de algoritmos

El peor caso corresponde a una cantidad $\mathcal{O}(n)$ de repetidos en primer y segundo apellido. Luego, el análisis de complejidad puede resumirse en

- Línea 1 en $\mathcal{O}(n \log(n))$
- Línea 2 en $\mathcal{O}(n)$
- **for** de línea 3 se ejecuta $\mathcal{O}(n)$ veces
 - Línea 4 en $\mathcal{O}(n \log(n))$
 - Línea 5 en $\mathcal{O}(n)$
 - **for** de línea 6 se ejecuta $\mathcal{O}(n)$ veces
 - Línea 7 en $\mathcal{O}(n \log(n))$

Con esto, la complejidad sería

$$\mathcal{O}(n \log(n) + n + n \cdot [n \log(n) + n + n \cdot (n \log(n))]) \Rightarrow \mathcal{O}(n^3 \log(n))$$

Ejemplo 2: Dividir para conquistar

Ejercicio (I1 P3 - 2022-2)

Dada una secuencia $A[0 \dots n-1]$ de números enteros, se define un **índice mágico** como un índice $0 \leq i \leq n-1$ tal que $A[i] = i$. Por ejemplo, en la siguiente secuencia

-7	3	2	5	-1	15	7	12	6	9	3
0	1	2	3	4	5	6	7	8	9	10

existen dos índices mágicos: el 2 y el 9.

Dada una secuencia $A[0 \dots n-1]$ ordenada, sin elementos repetidos e implementada como arreglo,

- proponga el pseudocódigo de un algoritmo que retorne un índice mágico en A si existe y que retorne `null` en caso contrario. Su algoritmo debe ser más eficiente que simplemente revisar el arreglo elemento por elemento, i.e. mejor que $\mathcal{O}(n)$.

Ejemplo 2: Dividir para conquistar

input : Arreglo $A[0, \dots, n-1]$, índices i, f

output: Índice mágico o null

```
1 Magic ( $A, i, f$ ):
2 begin
3     if  $(f - i) = 0$  :
4         begin
5             if  $A[i] = i$  :
6                 begin
7                     return  $i$ 
8                 end
9             end
10            return null
11        end
12         $p \leftarrow \lfloor (f - i)/2 \rfloor$ 
13        if  $A[p] = p$  :
14            begin
15                return  $p$ 
16            end
17        if  $A[p] > p$  :
18            begin
19                return Magic( $A, i, p - 1$ )
20            end
21        return Magic( $A, p + 1, f$ )
22    end
```

Sumario

Interrogación 1

Algoritmos de ordenación

Dividir para conquistar

Árboles de búsqueda

Un ejemplo de pruebas

Cierre

Recomendaciones finales

Para los algoritmos vistos en clase

- Comprender las demostraciones de correctitud
- Replicarlas por su cuenta
- Asegurarse de poder motivar el *¿por qué se plantea esta propiedad para demostrar?*
- Ser capaz de determinar su complejidad y casos